

Gestión de Datos de Países en Python

Filtros, ordenamientos y estadísticas

Universidad Tecnológica Nacional

Facultad Regional Mendoza

Tecnicatura Universitaria en Programación a Distancia

Programación I — Trabajo Práctico Integrador

Integrantes

Nombre	Legajo
Lautaro Martinez	54277
Luciano Pizarro	54249

July 17

Fecha de entrega: **Lunes 22 de junio de 2026**

Índice

1

Introducción

Objetivo y descripción general del sistema

2

Marco Teórico

Listas, diccionarios, funciones, condicionales, ordenamientos, estadísticas y CSV

3

Decisiones Técnicas

Arquitectura, estructura del proyecto, diagrama de flujo y validaciones

4

Dificultades y Conclusiones

Problemas encontrados, soluciones aplicadas y aprendizajes

5

Bibliografía y Enlaces

Fuentes consultadas y recursos del proyecto

1. Introducción

El presente Trabajo Práctico Integrador tiene como objetivo desarrollar una aplicación en Python capaz de gestionar información sobre países, aplicando los conceptos fundamentales de la materia **Programación I**: listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos y estadísticas básicas.

Carga de datos

Desde un archivo CSV con nombre, población, superficie y continente de cada país

Menú en consola

Listar, agregar, modificar, eliminar y buscar países de forma interactiva

Consultas avanzadas

Filtros, ordenamientos y estadísticas sobre el dataset de países

Persistencia

Los cambios se guardan en `países.csv` al salir del programa

2. Marco Teórico

2.1 Listas

Una **lista** en Python es una estructura de datos ordenada y mutable que permite almacenar una secuencia de elementos. Se utiliza cuando necesitamos guardar una colección de registros sobre los cuales vamos a iterar, filtrar u ordenar.

En este proyecto, la lista principal es `países`, que contiene todos los países cargados desde el CSV. Al listar se recorre completa, al filtrar se construye una nueva lista con los resultados, y al agregar o eliminar un país se modifica directamente con `append()` o `remove()`.

2.2 Diccionarios

Un **diccionario** es una estructura clave-valor que permite representar entidades con distintos atributos. A diferencia de una lista indexada por posición, se accede a cada dato mediante una clave identificatoria.

Cada país del sistema se modela como un diccionario con cuatro claves: `nombre`, `poblacion`, `superficie` y `continente`. La colección completa es una lista de diccionarios.

```
pais = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "America"  
}
```

2.3 Funciones

Las **funciones** permiten dividir el programa en bloques reutilizables, cada uno con una responsabilidad definida. Esto mejora la legibilidad, facilita el mantenimiento y evita repetir código.

El proyecto está organizado en módulos dentro de la carpeta `functions/`. El criterio aplicado fue **una función = una responsabilidad**.

2.4 Condicionales

Las **estructuras condicionales** (`if`, `elif`, `else`) permiten que el programa tome decisiones en función de condiciones booleanas. Son la base del menú, las validaciones de datos y el manejo de casos especiales.

Se utilizan para validar entradas del usuario, dirigir el flujo del menú y mostrar mensajes distintos cuando una búsqueda o filtro no arroja resultados.

Marco Teórico (continuación)



2.5 Ordenamientos

El **ordenamiento** consiste en reorganizar una secuencia de elementos según un criterio. En Python, la función `sorted()` devuelve una nueva lista ordenada y admite el parámetro `key` para definir por qué campo ordenar, y `reverse` para indicar orden ascendente o descendente.

El módulo `ordenamientos.py` implementa tres criterios: por nombre (alfabético), por población y por superficie. Internamente se usa `sorted()` con una función `lambda` que indica el campo de comparación.



2.6 Estadísticas básicas

Las **estadísticas básicas** permiten obtener información resumida de un conjunto de datos numéricos. En este proyecto se aplican operaciones como máximo, mínimo, promedio y conteo por categoría.

El módulo `estadisticas.py` calcula: el país con mayor y menor población (`max()` y `min()`), el promedio de población y superficie, y la cantidad de países por continente mediante un diccionario de conteo.



2.7 Archivos CSV

Un archivo **CSV** (Comma-Separated Values) almacena datos tabulares en texto plano, con campos separados por comas. Python incluye el módulo `csv` para leer y escribir este formato de manera sencilla.

Al iniciar, `cargar_países()` utiliza `csv.DictReader` para leer cada fila como un diccionario. Al salir, `guardar_países()` utiliza `csv.DictWriter` para persistir los cambios. Si el archivo no existe o contiene datos con formato incorrecto, el programa informa el error sin interrumpir la ejecución.

3. Decisiones Técnicas y Arquitectura

3.1 Estructura del proyecto

El proyecto sigue una **arquitectura modular**. El punto de entrada es `main.py`, que verifica las dependencias necesarias y ejecuta el menú principal. La lógica del sistema se organiza en la carpeta `functions/`:

Módulo	Responsabilidad
<code>menu.py</code>	Menú principal y enlace con cada funcionalidad
<code>datos.py</code>	Carga, guardado y gestión de países (ABM y búsqueda)
<code>filtros.py</code>	Filtrado por continente, población y superficie
<code>ordenamientos.py</code>	Ordenamiento por nombre, población y superficie
<code>estadisticas.py</code>	Cálculos estadísticos sobre el dataset
<code>utils.py</code>	Validaciones de entrada, formateo y funciones auxiliares
<code>estilos.py</code>	Tablas, títulos y formato visual en consola
<code>dependencias.py</code>	Verificación de librerías al iniciar el programa

Los datos persisten en `países.csv` en la raíz del proyecto. Las dependencias externas (`questionary` y `rich`) se declaran en `requirements.txt`.

3.2 Diagrama de Flujo del Sistema



El flujo del sistema comienza con la carga del archivo `países.csv` al iniciar. El usuario interactúa con el menú principal, selecciona una operación, el sistema la procesa y retorna al menú. Al elegir salir, los datos se persisten automáticamente en el CSV.

Capturas de ejecución

- **Menú principal:** `assets/menu-principal.png`
- **Listar países:** `assets/listar-paises.png`
- **Buscar país:** `assets/buscar-pais.png`

Recursos del proyecto

- Diagrama de flujo: `assets/diagrama_flujo_tpi.png`
- Dataset: `países.csv` en la raíz del proyecto
- Dependencias: `requirements.txt`

3.4 Validaciones Implementadas

El sistema incluye validaciones en distintos puntos para evitar errores y guiar al usuario hacia un uso correcto de la aplicación.

Campos vacíos y tipos de datos

Al agregar un país, no se permiten nombre, población, superficie ni continente vacíos. La población y la superficie deben ser enteros positivos; el nombre no puede contener números.

País duplicado o inexistente

Al agregar, se verifica que no exista otro país con el mismo nombre. Al modificar o eliminar, se informa si el nombre ingresado no está en la base de datos.

Máximo de intentos

Tras tres ingresos inválidos consecutivos, se ofrece volver al menú principal para evitar bucles infinitos de error.

CSV inexistente o con formato incorrecto

Se muestra un mensaje de error y el programa continúa con lista vacía o datos parcialmente cargados, sin interrumpir la ejecución.

Búsquedas y filtros sin resultados

Se informa claramente que no se encontraron países, indicando el criterio utilizado: término buscado, continente o rango de valores.

Rango inválido y dependencias faltantes

Al filtrar por población o superficie, el mínimo no puede ser mayor que el máximo. Al iniciar, se verifica que `questionary` y `rich` estén instaladas.

4. Dificultades y Conclusiones

4.1 Dificultades encontradas

Una de las dificultades principales fue **visualizar los datos en consola de forma clara**. Cuando el programa mostraba tablas y resultados usando únicamente `print()`, la información resultaba difícil de leer: columnas desalineadas, mucho ruido visual y poca diferenciación entre títulos, datos y mensajes de error.

4.2 Cómo se resolvieron

Para mejorar la presentación en consola, incorporamos **Rich**, que permite mostrar tablas alineadas, paneles de título, colores y estilos tipográficos. Centralizamos el formato visual en el módulo `estilos.py`, lo que permitió que listados, filtros, estadísticas y mensajes se muestren de manera ordenada y legible sin modificar la lógica de negocio de cada módulo.

4.3 Conclusiones y aprendizajes

Como conclusión del trabajo, aprendimos que una vez finalizado el MVP y las funcionalidades principales, resulta valioso pensar en el **usuario que va a utilizar el producto o servicio**. Mejorar la visualización y la experiencia de uso — incluso en una aplicación de consola — facilita la interacción y reduce errores. Separar la lógica del formato (datos vs. estilos) fue una decisión que nos permitió agregar esta capa de mejora sin reescribir el núcleo del programa.

- ✓ La separación entre lógica de negocio y presentación visual es un principio de diseño de software que aplica tanto a aplicaciones de consola como a sistemas de mayor escala.

5. Bibliografía y 6. Enlaces

Bibliografía / Webgrafía

Python Software Foundation. (s.f.).

Built-in Types — dict, list.

Documentación de Python 3.

<https://docs.python.org/3/library/stdtypes.html>

Python Software Foundation. (s.f.).

csv — CSV File Reading and Writing.

Documentación de Python 3.

<https://docs.python.org/3/library/csv.html>

Python Software Foundation. (s.f.).

Sorting HOW TO. Documentación de Python 3.

<https://docs.python.org/3/howto/sorting.html>

Textualize. (s.f.). *Rich — Render rich text, tables, progress bars, syntax highlighting, markdown and more.*

<https://rich.readthedocs.io/>

Questionary Contributors. (s.f.).

Questionary Documentation.

<https://questionary.readthedocs.io/>

Universidad Tecnológica Nacional.

(s.f.). *Campus Virtual UTN.*

<https://campusvirtual.frm.utn.edu.ar/>

Cátedra de Programación I. (2025). *Apuntes y material de estudio de la asignatura.* Universidad Tecnológica Nacional, Facultad Regional Mendoza.

6. Enlaces del Proyecto

Recurso	Enlace
Repositorio GitHub	https://github.com/lauto554/TPI
Video demostración	https://youtu.be/QkSYSbyWUUA
README del proyecto	https://github.com/lauto554/TPI#readme